



## Girls' Programming Network

*Candy Catcher!*

**FOR TUTOR EYES ONLY**

**This project was created by GPN Australia for GPN sites all around Australia!**

**This workbook and related materials were created by tutors at:**

Sydney and Melbourne



Girls' Programming Network

***If you see any of the following tutors don't forget to thank them!!***

**Writers**

Isabella Hogan

Angela Sojan

Sophie Lee-Goh

**Testers**

Amanda Hogan

# Scaffold

```
# <insert your name here>
# imports

# create constants

# print welcome

# make hero

# make candy

# add candy

# add game bits

# make timer (extension)

# start candy positions

# manage key controls
def on_key_down(key):
    # down up left right

    # stay on screen (extension)
    pass

# update everything
def update():
    # check collisions

    # win condition

    # timer (extension)
    pass

# draw everything to screen
def draw():
```

```
# empty screen

# draw candies

# draw hero

# draw text on screen

# runs everything
pgzrun.go()
```

# Part 1: Welcome to “Candy Catcher”

## 1.1: Who am I?

The code should look like this (no bonuses):

```
...  
# My name is ____  
# imports  
...
```

## 1.2: Welcome to ‘Candy Catcher’

The code should look like this (no bonuses):

```
...  
# print welcome  
print("""-----  
Welcome to Candy Catcher!  
-----  
Collect candies (1 point each)  
Avoid bombs (3 seconds off timer each)  
Reach 20 points to win!  
Press arrow keys to move.  
-----""")  
...
```

## 1.3: Setup Pygame Zero

The code should look like this (no bonuses):

```
...  
# imports  
import pgzrun  
  
# runs everything  
pgzrun.go()  
...
```

## 1.4: Make a screen

The code should look like this (no bonuses):

```
...
# set grid size
WIDTH = 640
HEIGHT = 480
...
```

## Full solution after Part 1

The code should look like this (no bonuses):

```
# My name is ____
# imports
import pgzrun

# set grid size
WIDTH = 640
HEIGHT = 480

# print welcome
print("""-----
Welcome to Candy Catcher!
-----
Collect candies (1 point each)
Avoid bombs (3 seconds off timer each)
Reach 20 points to win!
Press arrow keys to move.
-----""")

# make hero

# create constants

# make candies

# add candies

# loop candies

# add game bits

# make timer
```

```
# manage key controls

    # down up left right

# update everything
def update():
    # check collisions

    # win condition

    # timer

    # add a new candy

    pass

# draw everything to screen
def draw():
    # empty screen

    # draw candies

    # draw hero

    # draw text on screen
    pass

# runs everything
pgzrun.go()
```

# Part 2: Holding out for a hero

## 2.1: Creating the hero

The code should look like this (no bonuses):

```
...
# make hero
hero = Actor("hero")
...
```

## 2.2: Drawing the hero to the screen

The code should look like this (no bonuses):

```
...

# draw everything to screen
def draw():
    # empty screen

    # draw candies

    # draw hero
    hero.draw()
...
```

Note: students have been asked to remove “pass” from the draw function. “pass” is required in the scaffold, as Python does not allow empty function declarations. If “pass” is left in the code, however, that is fine and will not impact the code functionality.

## Full solution after Part 2

The code should look like this (no bonuses):

```
# imports
import pgzrun

# set grid size
WIDTH = 640
HEIGHT = 480
```

```

# print welcome
print("""-----
Welcome to Candy Catcher!
-----

Collect candies (1 point each)
Avoid bombs (3 seconds off timer each)
Reach 20 points to win!
Press arrow keys to move.
-----""")

# make hero
hero = Actor("hero")

# create constants

# make candies

# add candies

# loop candies

# add game bits

# make timer

# manage key controls

    # down up left right

# update everything
def update():
    # check collisions

    # win condition

    # timer

    # add a new candy

    pass

# draw everything to screen

```

```
def draw():  
    # empty screen  
  
    # draw candies  
  
    # draw hero  
    hero.draw()  
  
    # draw text on screen  
    pass  
  
# runs everything  
pgzrun.go()
```

# Part 3: I like to move it, move it!

## 3.1: Adding the maths

The code should look like this (no bonuses):

```
...
# set grid size
square = hero.width
num_squares_width = WIDTH // square
num_squares_height = HEIGHT // square

...
```

## 3.2: Moving the hero down

The code should look like this (no bonuses):

```
...
# manage key controls
def on_key_down(key):

    # down up left right
    hero.bottom = hero.bottom + square
...
```

## 3.3: Moving down when we want to

The code should look like this (no bonuses):

```
...
# manage key controls
def on_key_down(key):

    # down up left right
    if key == keys.DOWN:
        hero.bottom = hero.bottom + square

...
```

### 3.4: Adding other moves

The code should look like this (no bonuses):

```
...
# manage key controls
def on_key_down(key):

    # down up left right
    if key == keys.DOWN:
        hero.bottom = hero.bottom + square
    if key == keys.UP:
        hero.top = hero.top - square
    if key == keys.LEFT:
        hero.left = hero.left - square
    if key == keys.RIGHT:
        hero.right = hero.right + square
...
```

### 3.5: Painting it black

The code should look like this (no bonuses):

```
...
def draw():

    # empty screen
    screen.fill("black")
...
```

### Full solution after Part 3

The code should look like this (no bonuses):

```
# imports
import pgzrun

# set grid size
WIDTH = 640
HEIGHT = 480

# print welcome
print("""-----
Welcome to Candy Catcher!
-----
Collect candies (1 point each)
Avoid bombs (3 seconds off timer each)
Reach 20 points to win!
Press arrow keys to move.
-----""")

# make hero
hero = Actor("hero")

# create constants
square = hero.width
num_squares_width = WIDTH // square
num_squares_height = HEIGHT // square

# make candies

# add candies

# loop candies

# add game bits

# make timer

# manage key controls
def on_key_down(key):

    # down up left right
    if key == keys.DOWN:
        hero.bottom = hero.bottom + square
    if key == keys.UP:
        hero.top = hero.top - square
```

```

    if key == keys.LEFT:
        hero.left = hero.left - square
    if key == keys.RIGHT:
        hero.right = hero.right + square

# update everything
def update():
    # check collisions

    # win condition

    # timer

    # add a new candy

    pass

# draw everything to screen
def draw():
    # empty screen
    screen.fill("black")

    # draw candies

    # draw hero
    hero.draw()

    # draw text on screen
    pass

# runs everything
pgzrun.go()

```

### Bonus 3.6: Keeping the hero on screen

```

...
# down up left right
    if key == keys.DOWN:
        hero.bottom = min(hero.bottom + square, HEIGHT)
    if key == keys.UP:
        hero.top = max(hero.top - square, 0)
    if key == keys.LEFT:
        hero.left = max(hero.left - square, 0)
    if key == keys.RIGHT:
        hero.right = min(hero.right + square, WIDTH)
...

```

# Part 4: I want candy!

## 4.1: Creating a candy

The code should look like this (no bonuses):

```
...  
# make candy  
candy = Actor("candy")  
  
...
```

## 4.2: Putting it on the screen in a random position

The code should look like this (no bonuses):

```
...  
# imports  
import pgzrun  
import random  
  
# start candy positions  
candy.top = random.randint(0, num_squares_height - 1) * square  
candy.left = random.randint(0, num_squares_width - 1) * square  
  
# draw candies  
    candy.draw()  
  
...
```

#### 4.3: Adding more candies

The code should look like this (no bonuses):

```
...
# loop candies
candies = []
for i in range(3):
    candy = Actor("candy")
    candy.top = random.randint(0, num_squares_height - 1) * square
    candy.left = random.randint(0, num_squares_width - 1) * square
    candies.append(candy)

# start candy positions

# draw candies
    for candy in candies:
        candy.draw()
...
```

#### 4.4: Cleaning up the code

The code should look like this (no bonuses):

```
...
# add candy
def add_candy():
    candy = Actor("candy")
    candies.append(candy)
    candy.top = random.randint(0, num_squares_height - 1) * square
    candy.left = random.randint(0, num_squares_width - 1) * square

# make candy
candies = []
for i in range(3):
    add_candy()

...
```

#### 4.5. Collecting candies

The code should look like this (no bonuses):

```
...
# update everything
def update():

    # check collisions
    for candy in candies:
        if hero.colliderect(candy):
            candies.remove(candy)
...
```

#### Full solution after Part 4

The code should look like this (no bonuses):

```
# imports
import pgzrun

# set grid size
WIDTH = 640
HEIGHT = 480

# print welcome
print("""-----
Welcome to Candy Catcher!
-----
Collect candies (1 point each)
Avoid bombs (3 seconds off timer each)
Reach 20 points to win!
Press arrow keys to move.
-----""")

# make hero
hero = Actor("hero")

# create constants
square = hero.width
num_squares_width = WIDTH // square
num_squares_height = HEIGHT // square

# make candies

# add candies
```

```

def add_candy():
    candy = Actor("candy")
    candies.append(candy)
    candy.top = random.randint(0, num_squares_height - 1) * square
    candy.left = random.randint(0, num_squares_width - 1) * square

# loop candies
candies = []
for i in range(3):
    add_candy()

# add game bits

# make timer

# manage key controls
def on_key_down(key):

    # down up left right
    if key == keys.DOWN:
        hero.bottom = hero.bottom + square
    if key == keys.UP:
        hero.top = hero.top - square
    if key == keys.LEFT:
        hero.left = hero.left - square
    if key == keys.RIGHT:
        hero.right = hero.right + square

# update everything
def update():
    # check collisions
    for candy in candies:
        if hero.colliderect(candy):
            candies.remove(candy)

    # win condition

    # timer

    # add a new candy

    pass

# draw everything to screen
def draw():
    # empty screen
    screen.fill("black")

```

```
# draw candies
for candy in candies:
    candy.draw()
# draw hero
hero.draw()

# draw text on screen
pass

# runs everything
pgzrun.go()

...
```

# Part 5: Moooooaaaaarrrrr!

## 5.1: What's the time Mr Wolf?

The code should look like this (no bonuses):

```
...
# make timer
frames = 0

...
```

## 5.2: Doing some maths

The code should look like this (no bonuses):

```
...
# update everything
def update():
    # check collisions
    global frames
    for candy in candies:
        if hero.colliderect(candy):
            candies.remove(candy)

# timer
frames = frames + 1

...
```

## 5.3: More Candy!

The code should look like this (no bonuses):

```
...

# timer
frames = frames + 1
# add a new candy
if frames % 180 == 0:
    add_candy()

...
```

## Full solution after Part 5

The code should look like this (no bonuses):

```
# imports
import pgzrun

# set grid size
WIDTH = 640
HEIGHT = 480

# print welcome
print("""-----
Welcome to Candy Catcher!
-----
Collect candies (1 point each)
Avoid bombs (3 seconds off timer each)
Reach 20 points to win!
Press arrow keys to move.
-----""")

# make hero
hero = Actor("hero")

# create constants
square = hero.width
num_squares_width = WIDTH // square
num_squares_height = HEIGHT // square

# make candies

# add candies
def add_candy():
    candy = Actor("candy")
    candies.append(candy)
    candy.top = random.randint(0, num_squares_height - 1) * square
    candy.left = random.randint(0, num_squares_width - 1) * square
# loop candies
candies = []
for i in range(3):
    add_candy()

# add game bits

# make timer
frames = 0
# manage key controls
def on_key_down(key):
```

```

# down up left right
if key == keys.DOWN:
    hero.bottom = hero.bottom + square
if key == keys.UP:
    hero.top = hero.top - square
if key == keys.LEFT:
    hero.left = hero.left - square
if key == keys.RIGHT:
    hero.right = hero.right + square

# update everything
def update():
    # check collisions
    for candy in candies:
        if hero.colliderect(candy):
            candies.remove(candy)

    # win condition

    # timer
    frames = framesv+ 1
    # add a new candy
    if frames % 180 == 0:
        add_candy()

    pass

# draw everything to screen
def draw():
    # empty screen
    screen.fill("black")

    # draw candies
    for candy in candies:
        candy.draw()

    # draw hero
    hero.draw()

    # draw text on screen
    pass

# runs everything
pgzrun.go()

```

# Part 6: Winner Winner!

## 6.1: Any more candies?

The code should look like this (no bonuses):

```
...
# win condition
    if len(candies) == 0:
        print("You won")

...
```

## 6.2: Stopping the game

The code should look like this (no bonuses):

```
...
# add game bits
win = False

# manage key controls
def on_key_down(key):
    if win == True:
        return

def update():
    global frames, win
    # check collisions
    if win == True:
        return
    for candy in candies:
        if hero.colliderect(candy):
            candies.remove(candy)

# win condition
    if len(candies) == 0:
        print("You won")
        win = True

...
```

### 6.3: Draw timer on screen

The code should look like this (no bonuses):

```
...
# draw text on screen
    if win == True:
        screen.fill("green")
        screen.draw.text(("You won!!
"), (0,5), color="white", fontsize=20)
...
```

### Full solution after Part 6

The code should look like this (no bonuses):

```
# imports
import pgzrun

# set grid size
WIDTH = 640
HEIGHT = 480

# print welcome
print("""-----
Welcome to Candy Catcher!
-----
Collect candies (1 point each)
Avoid bombs (3 seconds off timer each)
Reach 20 points to win!
Press arrow keys to move.
-----""")

# make hero
hero = Actor("hero")

# create constants
square = hero.width
num_squares_width = WIDTH // square
num_squares_height = HEIGHT // square

# make candies

# add candies
def add_candy():
    candy = Actor("candy")
    candies.append(candy)
```

```

    candy.top = random.randint(0, num_squares_height - 1) * square
    candy.left = random.randint(0, num_squares_width - 1) * square
# loop candies
candies = []
for i in range(3):
    add_candy()

# add game bits
win = False

# make timer
frames = 0
# manage key controls
def on_key_down(key):
    if win == True:
        return

    # down up left right
    if key == keys.DOWN:
        hero.bottom = hero.bottom + square
    if key == keys.UP:
        hero.top = hero.top - square
    if key == keys.LEFT:
        hero.left = hero.left - square
    if key == keys.RIGHT:
        hero.right = hero.right + square

# update everything
def update():
    global frames, win
    if win == True:
        return
    # check collisions
    for candy in candies:
        if hero.colliderect(candy):
            candies.remove(candy)

    # win condition
    if len(candies) == 0:
        print("You won")
        win = True
    # timer
    frames = frames + 1
    # add a new candy
    if frames % 180 == 0:
        add_candy()

```

```
    pass

# draw everything to screen
def draw():
    # empty screen
    screen.fill("black")

    # draw candies
    for candy in candies:
        candy.draw()

    # draw hero
    hero.draw()

    # draw text on screen
    pass

# runs everything
pgzrun.go()
```

# Part 7: Extension 1 - What's the time Mr Wolf!

## 7.1: 3, 2, 1...

The code should look like this (no bonuses):

```
...
# make timer
frames = 1800

# timer
    frames = frames - 1

...
```

## 7.2: Time on the screen

The code should look like this (no bonuses):

```
...
# make timer
frames = 1800
timer = frames // 60

# timer
    frames = frames - 1
    # add a new candy
    if frames % 180 == 0:
        add_candy()

    timer = frames // 60

screen.draw.text(("Timer:
"+str(timer)), (0,5), color="white", fontsize=20)
...
```

### 7.3: Stopping the game

The code should look like this (no bonuses):

```
...
# manage key controls
def on_key_down(key):
    if win == True or lose == True:
        Return

# update everything
def update():
    global frames, win, timer, lose
    # check collisions
    if win == True or lose == True:
        Return

timer = frames // 60

    if timer <= 0:
        lose = True
...

```

### 7.4: Game over pt. 2

The code should look like this (no bonuses):

```
...
# draw text on screen
    if lose == True:
        screen.fill("red")
        screen.draw.text(("You lost!!
"), (0,5),color="white",fontsize=20)

```

# Part 8: Extension 2 - A better game

## 8.1: Keeping count

The code should look like this (no bonuses):

```
...
# add game bits
win = False
lose = False
score = 0

def update():
    global frames, win, timer, lose, score

    if hero.colliderect(candy):
        candies.remove(candy)
        score = score + 1...
```

## 8.2: Showing the score

The code should look like this (no bonuses):

```
...
# add game bits
screen.draw.text(("Score:
"+str(score)), (0,15), color="white", fontsize=20)
```

## Full solution after Extensions

The code should look like this:

```
...
# imports
import pgzrun
import random

# set grid size
WIDTH = 640
HEIGHT = 480

# print welcome
print("""-----
Welcome to Candy Catcher!
```

```

-----
Collect candies (1 point each)
Avoid bombs (3 seconds off timer each)
Reach 20 points to win!
Press arrow keys to move.
-----"""

# make hero
hero = Actor("hero")

# create constants
square = hero.width
num_square_width = WIDTH // square
num_square_height = HEIGHT // square

# make candies
candy = Actor("candy")

# add candies
def add_candy():
    candy = Actor("candy")
    candies.append(candy)
    candy.top = random.randint(0, num_square_height - 1) * square
    candy.left = random.randint(0, num_square_width - 1) * square

# loop candies
candies = []
for i in range(3):
    add_candy()

# add game bits
win = False
lose = False
score = 0

# make timer
frames = 1800
timer = frames // 60
# start candy positions

# manage key controls
def on_key_down(key):
    if win == True or lose == True:
        return
    # down up left right
    if key == keys.DOWN:

```

```

        hero.bottom = min(hero.bottom + square, HEIGHT)
    if key == keys.UP:
        hero.top = max(hero.top - square, 0)
    if key == keys.LEFT:
        hero.left = max(hero.left - square, 0)
    if key == keys.RIGHT:
        hero.right = min(hero.right + square, WIDTH)

# update everything
def update():
    global frames, win, timer, lose, score
    # check collisions
    if win == True or lose == True:
        return
    for candy in candies:
        if hero.colliderect(candy):
            candies.remove(candy)
            score = score + 1

    # win condition
    if len(candies) == 0:
        print("You won")
        win = True

    # timer
    frames = frames - 1
    # add a new candy
    if frames % 180 == 0:
        add_candy()

    timer = frames // 60

    if timer <= 0:
        lose = True
    pass

# draw everything to screen
def draw():
    # empty screen
    screen.fill("black")

    # draw candies
    for candy in candies:
        candy.draw()

    # draw hero
    hero.draw()

```

```
# draw text on screen
if win == True:
    screen.fill("green")
    screen.draw.text(("You won!!"), (0,5), color="white", fontsize=20)

# draw text on screen
if lose == True:
    screen.fill("red")
    screen.draw.text(("You lost!!"), (0,5), color="white", fontsize=20)

    screen.draw.text(("Timer: "+str(timer)), (0,5), color="white", fontsize=20)
    screen.draw.text(("Score: "+str(score)), (0,15), color="white", fontsize=20)

# runs everything
pgzrun.go()
...
```